Semisupervised-Learning-Based Security to Detect and Mitigate Intrusions in IoT Network

Nagarathna Ravi, *Student Member, IEEE*, and S. Mercy Shalinie, *Senior Member, IEEE*

***Abstract*—Our world is moving toward an Internet of Things (IoT) era by connecting billions of IoT. There are several security loopholes in the IoT network. Intrusion can lead to performance degradation and pose a threat to data security. Hence, there is a need for a method to detect intrusion in the IoT networks. Existing solutions use supervised-learning-based intrusion detection methods that need a huge labeled data set for better accuracy. It is not easy to source out a huge labeled data set because the size of the IoT network is huge. To overcome some of the impediments in the existing solutions, we propose a novel SDRK machine learning (ML) algorithm to detect intrusion. SDRK leverages supervised deep neural networks (DNNs) and unsupervised clustering techniques. The intrusion detection and mitigation algorithms are placed in the fog nodes that are between IoT and cloud layers. We test our proposed methodology against the data deluge (DD) attack in the testbed. The SDRK model is tested on the benchmark NSL-KDD data set. We compare the results with state-of-the-art solutions. When testing with the NSL-KDD data set, we find that SDRK detects the attacks with improved accuracy of 99.78%.**

***Index Terms*—Data deluge (DD) attack, fog computing, Internet of Things (IoT), intrusion, security, semisupervised learning.**

List of Abbreviation

DD	Data deluge attack.
DFNN	Deep feedforward neural network.
K	Number of clusters.
R	Number of runs.
RRS	Repeated random sampling.
SDRK	SSML DFNN-RRS-K-means.
SSML	Semisupervised ML.
ζ	Error probability.

I. Introduction

INTERNET of Things (IoT) is resource-constrained devices that sense the environment and transmit the sensed data. IoT finds its application in latency-critical automated use cases, such as healthcare, autonomous driving, mining, chemical factory, power grid, emergency alarm system, autistic children aid system, etc. [1]. There are security loopholes in the IoT network which can stop the users from accepting the IoT use cases [2]. Dyn lost 8% of customers due to the Mirai attack [1]. Security breaches can create performance degradation. The degraded service in healthcare may be life threatening [3]. The Food and Drug Association in the U.S. has affirmed the security risk due to unauthorized access to medical IoT [1]. Thus, it is necessary to provision rich security modules that can identify the attacks early, so that the use cases can bounce back from the effects of the attack and resume the normal operation at the earliest. But the resource constraints in IoT make it difficult to provision rich security modules in IoT. The diverse IoT brands and user requirements also make it difficult to implement cross-device security solutions [4].

The attackers' strategy is improving greatly. Thus, intrusion detection systems based on attack signatures cannot provide efficient security. A machine learning (ML) approach can learn the nooks and corners of the threats [3] and identify even small malicious mutations in traffic [5]. Hence, an appropriate ML model with low attack detection time and better levels of accuracy is needed for IoT networks. Some existing solutions use supervised ML [6], [7]. IoT is a huge network and so, it generates a large volume of data. It is laborious to label the big data set and is prone to human error. Some existing solutions use unsupervised ML [8]. Unsupervised ML results in low attack prediction accuracy [7]. The fact that intrusion in IoT leads to the degraded performance, it is not easy to provision security modules in IoT, and there are issues in using supervised and unsupervised ML for IoT motivates us to find suitable intrusion detection and mitigation methodology. We focus on developing a methodology that will have better attack detection time and better accuracy levels. While probing the ML models, we find that the SSML can avoid the pitfalls of supervised and unsupervised ML and provide improved accuracy of attack detection [9]. So, we propose a novel SSML model named deep feedforward neural network-repeated random sampling-K-means (SDRK) model for detecting intrusion in IoT networks. We give a mitigation methodology to mitigate the attack. The whole methodology is deployed in the fog nodes of fog–IoT architecture.

The key contributions of this article are as follows.

1) We propose a novel SDRK model to detect intrusion in IoT networks.
2) We probe into the hardness of the K-means algorithm in provisioning security for IoT. We find it to be NP-complete. We propose a novel randomized solution named repeated random sampling-K-means (RRS-K-means) for the NP-complete problem. We prove the lower bound of the error probability of RRS-K-means.

Manuscript received March 17, 2020; revised April 21, 2020; accepted May 5, 2020. Date of publication May 8, 2020; date of current version November 12, 2020. *(Corresponding author: Nagarathna Ravi.)*

The authors are with the Department of Computer Science and Engineering, Thiagarajar College of Engineering, Madurai 625015, India (e-mail: nagarathna@student.tce.edu; shalinie@tce.edu).

Digital Object Identifier 10.1109/JIOT.2020.2993410

2327-4662 © 2020 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:59:33 UTC from IEEE Xplore. Restrictions apply.

3) We assess our methodology in IoT–fog testbed against the DD attack to verify its working in the real network.
4) We evaluate SDRK with the benchmark NSL-KDD data set and compare the results with state-of-the-art solutions. SDRK shows less testing time and improved accuracy than other models. This shows that SDRK is a better model for latency-critical IoT use cases.

The remainder of this article is organized as follows. Section II provides an overview of threats in the IoT network and discusses the different IoT architectures. Section III provides a gist of the related works. Section IV provides details of the proposed methodology. Section V provides the results obtained during the evaluation and Section VI provides the concluding notes.

II. Analysis of Threats in IoT and Suitable Architecture

A. Analysis of Threats in IoT

A wide range of attacks has been identified in the IoT threat taxonomy. In this section, we give a concise description of some of the attacks. We divide the section into three sections based on the core security objectives, namely, confidentiality, availability, and integrity.

Attacks That Breach Confidentiality: Attackers can get unauthorized access to the IoT network in various ways. The Sybil attack compromises the IoT to gain access [10]. Satan, IPsweep, Portsweep, Nmap, Warezclient, and Guess_passwd attacks try to find the weakness in the network and gain access either by finding out the credentials of the IoT or by forcing the IoT to download the illegal software [6]. The dictionary attack does a brute force attack on the IoT to find out the credentials and then malware is downloaded in the IoT [11]. In such types of attacks, the compromised nodes prevent legitimate traffic or generate abnormal traffic [10]. In other types of attacks that try to gain access, abnormal traffic comes in the IoT network. For instance, the dictionary attack repeatedly hits the IoT with several combinations of credentials. Hence, a careful analysis of the traffic by an apt ML model will aid in detecting the attacks that try to breach confidentiality.

Attacks That Breach Availability: Attackers can make the IoT network service unavailable to the users in several ways. Distributed Denial of Service (DDoS), DD, packet flooding, device capture, battery draining, carousel, stretch, and jamming attacks aim to bring down the IoT network [1]. The attacks targeting the availability can be primarily generated in two ways. The first way involves creating a flood of traffic. The second way involves blocking a part of IoT traffic. Thus, the network will witness a surge or a drop in normal traffic. Hence, an analysis of traffic by the ML model will prevent the IoT network from going down.

Attacks That Breach Integrity: Attackers can breach the integrity by injecting false packets or forwarding selective packets or modifying the IoT firmware. Injecting false data will introduce abnormal traffic. Forwarding selective packets by a particular route or dropping some packets will also be different from the normal traffic. For instance, forwarding selective traffic via a route will create a surge in traffic in that route. Modification in the IoT firmware requires introducing malware traffic. Thus, the analysis of traffic by the ML model will aid in detecting the integrity breach.

From the above discussion, it can be said that an apt ML model trained with the traffic traces observed in the IoT network can detect most of the attacks.

B. Analysis of Suitable IoT Architecture

Various architectures are proposed for IoT networks. The prominent architectures are software-defined network (SDN) [12], information-centric network (ICN) [13], and fog [14].

SDN is a centralized architecture which makes the data plane as mute followers of the rules of the central controller. SDN–IoT architecture has several advantages like the ease of reconfiguration, flexible addition of features, etc. [12]. But when the controller fails, the IoT network also fails [15]. Even if we have a backup plan to handle the failure, the switch having the flow rules has a limited capacity [15]. As the IoT network is huge, the count of unique packet flows generated will exceed the limited buffer capacity of a switch and, there will be frequent switch buffer overflow. When there is no rule for the packet, the switch requests the controller to set rules. Thus, there is a communication overhead due to the control traffic between the IoT switch and the controller [15].

ICN is a content-centric architecture. The accessing of the packets is done using names and not with IP addresses. Since the content from IoT is important rather than its location, ICN has been proposed as a better architecture for IoT [13]. A major drawback of ICN is that the content is transferred only when a request is generated for the data. This can be a bottleneck when IoT sends periodic data because there are fewer chances of getting the current data when taking into account the time for traversal of request packets [16].

In fog computing, there are fog nodes distributed between the IoT and cloud layer. The cloud offloads some processing modules, such as storage, data computation, security, etc., to the fog nodes. This provides advantages, such as reduced latency, improved service efficiency, etc. [3]. Fog nodes can detect and prevent the attack locally emanating from the IoT domain beneath it. The fog node prevents the attack propagation up to the cloud layer and the disruption of services in the cloud. It also saves the bandwidth of the cloud. Moreover, the fog node has the location details of the IoT encompassed by its domain. This provides added advantage in locating the malicious IoT or the intruder in the network. If the security feature is incorporated in the cloud layer, tracing the attacker will be like searching a pin in a huge ocean. When compared to SDN, there is no control traffic overhead in fog. When compared to ICN, fog does not disturb the normal working of the IoT. The major challenge in fog is the efficient resource utilization-aware deployment of fog nodes. But there are several solutions to address this challenge in [14] and [17]. Owing to these advantages, we propose the placement of our methodology in the fog nodes assuming that the fog nodes are secure. (Efficient resource utilization-aware deployment of fog nodes is not under the scope of this article.)

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:59:33 UTC from IEEE Xplore. Restrictions apply.

TABLE I
RELATED WORK

Researcher	Detection	Deployment	Mitigation	Dataset	Pros and Cons
Kozik et al. [18]	ELM	Edge & cloud	-	CTU	Pros- Short training time; Cons- Overfitting issue
Prabavathy et al. [3]	OS-ELM	Fog node	-	NSL-KDD	Pros- Short training time; Cons- Overfitting issue
Rathore et al. [9]	ELM Fuzzy C Means	Fog node	-	NSL-KDD	Pros- Short training time; Cons- Overfitting issue; Apt confidence threshold should be set to get a better model
Diro et al. [6]	Deep learning	Fog node	-	NSL-KDD	Pros- Deep learning learns well even in resource-constrained network; Model is updated frequently [6]; Cons- Scalability issues in real world deployment
Pajouh et al. [7]	LDA, Naive Bayes, K-NN	-	-	NSL-KDD	Pros- Lightweight model; Cons- Scalability issues in real world deployment, Not discussed about deployment
Zhang et al. [8]	Genetic algorithm & DBN	-	-	NSL-KDD	Pros- Adaptive generation of hidden layers and neurons; Cons- DBN may not work efficiently in cases of uncertain input, Not discussed about deployment
Jan et al. [10]	SVM	-	-	CICIDS2017	Pros- Lightweight model; Cons- Single attribute is not sufficient, Not discussed about deployment
Li et al. [21]	Bat algorithm & Random Forest	SDN controller	-	KDD	Pros- Overcomes the performance degradation issues in Random Forest; Cons- Central control is prone to failure

III. RELATED WORK

Various researchers have done works in the realm of intrusion in IoT. Table I illustrates the essence of related works, and a short description is provided in this section.

Kozik *et al.* [18] proposed a distributed attack detection model based on the extreme learning machine (ELM). The training is done at the cloud and, the trained model is deployed at the edge devices. The model is distributed to the edge nodes using the map-reduce technique. The traffic is captured and, features, such as flow count, bytes transferred, communication time, etc., are extracted and given as input to the model.

Prabavathy *et al.* [3] proposed an online sequential ELM (OS-ELM)-based approach. The training and detection are both performed in the fog node. When malicious activity is detected, the findings are sent to the cloud server. This will help the admin to analyze the strategies of the attacker and formulate an appropriate intrusion response methodology.

Rathore and Park [9] proposed an SSML model which is a combination of ELM and fuzzy *C* means (FCM). Initially, ELM is applied to the labeled data set. The labeled and unlabeled data set is given as input to FCM. The confidence in the prediction of unlabeled data is calculated. If the confidence is greater than a threshold, it is placed in the labeled data set. Else, it is reclustered and the training process is repeated. The trained model is put in the fog nodes.

Though ELM-based models proposed in [3], [9], and [18] possess the merit of short training time, ELM's overfitting issue reduces the efficiency of detection in the real-world deployment [19].

Diro and Chilamkurti [6] proposed to use deep learning to detect attacks. The framework has two levels of fog. The master node at the top level trains and puts the model in the fog nodes. The second-level fog nodes send model updates to the master node. The master node updates the global model and disseminates the updates to the fog nodes. This aids in having an updated model in all the fog nodes. In real-time networks, when a large volume of traffic is generated, the sending and receiving of updates may turn out to be a cumbersome process.

Pajouh *et al.* [7] presented the TDTC model which has principal component analysis (PCA) and linear discriminant analysis (LDA) to extract the features that are capable of improving the efficiency of the attack classification. The certainty factor *K*-nearest neighbor (*K*-NN) and Naive Bayes (NB) are used to detect the attack. The NB detects the attack. The findings of the NB is further refined using the *K*-NN.

As already said, a supervised approach in [6] and [7] cannot scale up efficiently in a huge IoT network.

Zhang *et al.* [8] proposed a model that combines the genetic algorithm and the unsupervised deep belief network (DBN) to detect the intrusion. A set of values for the count of hidden layers and neurons in the hidden layers are generated. The appropriate parameter values are found using the fitness function. The test data set is input to the DBN with the found layer parameters. The parameters that give the best performance are retained and, the trained model is generated. If performance is not satisfactory, a new set of parameter values are generated and, the operation is repeated. DBN may not work efficiently in cases of uncertain input due to its inherent trait of single bottom-up pass [20]. IoT is constrained and lag in provisioning certain input all the time. DBN may overlook the uncertain data from the IoT. The authors have not provided details of the deployment place in the IoT network.

Jan *et al.* [10] proposed an intrusion detection model based on the support vector machine (SVM) to detect DDoS attacks alone. The authors showed that instead of using more attributes for attack classification, a single attribute named packet arrival rate is sufficient to detect DDoS. During traffic bursts, the packet arrival rate can rise above the normal packet arrival rate. In such cases, a single attribute cannot be used for detecting the attack. The authors have not provided details of the deployment place in the IoT network.

Li *et al.* [21] presented a two-stage intrusion detection model based on the bat algorithm and random forest for software-defined IoT. The random forest suffers from performance degradation due to unwanted features. The authors used the bat algorithm to select the appropriate features. The attack is detected using the random forest. The

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:59:33 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DATA SET FEATURE DESCRIPTION

Feature	Calculation	Notation
Packet count	#(Packets recorded in time window)	PC
Link bandwidth utilization	$\frac{\sum_{i=1}^{i=PC}(Packetsize)_i}{Bandwidth}$	LB
Average	$\frac{\sum_{i=1}^{i=PC}(Packetsize)_i}{PC}$	A
Weighted moving average	$\frac{\sum_{i=1}^{i=t}(s)^i(A)_i}{t}$ weights were initialized based on geometric series	WA
Standard deviation	$\sqrt{\frac{\sum_{i=1}^{PC}(Packetsize_i - A)^2}{(PC-1)}}$	SD
Magnitude [26]	$\sqrt{A^2 + A(t-1)^2}$	M
Radius [26]	$\sqrt{SD^{2\,2} + SD(t-1)^{2\,2}}$	R
Covariance	$\frac{\sum_{i=1}^{n}(T_i - A)(T(t-1)_i - A(t-1))}{n-1}$ n denotes the count of common source IP that have sent data in the current & previous time window. T denotes the total size of packets sent by i^{th} source IP.	CV
Correlation [26]	$\frac{CV}{SD \times SD(t-1)}$	CR
Covariance	$\frac{\sum_{i=1}^{n}(T_i - A)(T(t-1)_i - A(t-1))}{n-1}$ n denotes the count of common source MAC that have sent data in the current & previous time window. T denotes the total size of packets sent by i^{th} source MAC.	CV_1
Correlation	$\frac{CV}{SD \times SD(t-1)}$	CR_1
Entropy	$-\sum_{i=1}^{PC}(\frac{packetsize_i}{PC} \times \log_2 \frac{packetsize_i}{PC})$	E
Information distance	$-\sum_{i=1}^{PC}(\frac{packetsize_i}{PC} \times \log_2(\frac{packetsize_i}{PC * avg(prob_j)}))$, where $avg(prob_j) = \sum_{k=1}^{PC(t-j)}(\frac{packetsize(t-j)_k}{PC(t-j)})$	ID_j

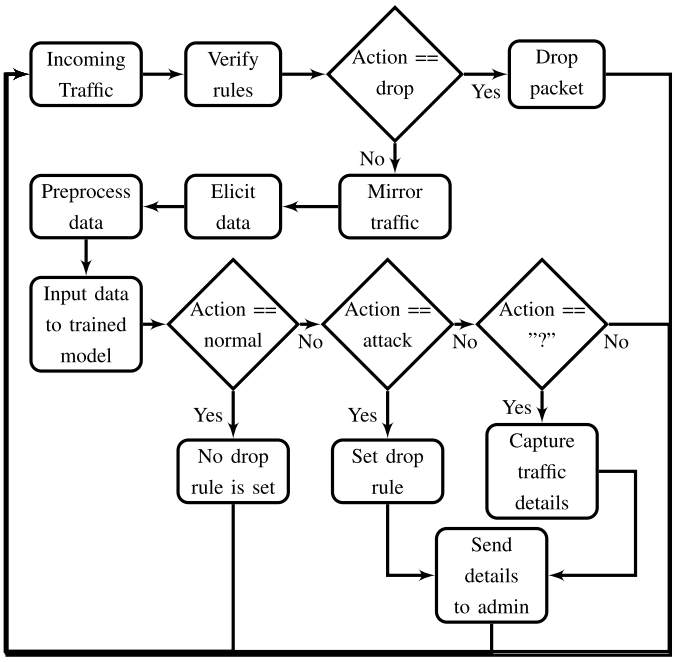


Fig. 1. Proposed methodology.

output is chosen from the trees using the weighted voting method. The weights are assigned to the trees based on the accuracy of the trees in detecting the attack. The authors deployed the model in the central SDN controller. The central model may collapse when the controller's data processing load increases [22].

Our proposed methodology strives to overcome some of the issues in the related works. We deploy the traffic data collection module, trained SDRK module, and attack mitigation module in the fog nodes distributed across the IoT networks. This avoids the central failure issue in [21]. To avoid the issues in the supervised learning models [6], [7], we use an SSML model to detect the intrusion. Our SDRK learning model combines the power of supervised and unsupervised learning models rather than using the unsupervised learning model alone as in [8]. As the security functionality is offloaded to the fog node, the diverse and resource-constrained IoT need not possess the capacity to provide security.

IV. PROPOSED METHODOLOGY

The methodology has three modules running together as shown in Fig. 1. The modules are presented in detail as follows.

A. Data Collection

The fog node is set as the gateway for the IoT network. This enables the data traffic to flow through the fog node before reaching the server. Initially, the incoming traffic is verified against the access control list (ACL) rules in the fog node. If the rule is drop, the traffic is dropped. Else, the traffic is mirrored and necessary features, such as packet size, packet count, etc., that will help in the identification of the attack are extracted. For illustration, we provide the details of features of an IoT attack named DD attack in the evaluation, Section V-A in Table II. (To provide the data set after presenting the DD attack model, we describe it in the evaluation, Section V.) This data set has 22 statistical attributes obtained by processing the IoT pcap traffic. Similarly, the features that can effectively recognize other security breaches are extracted from the pcap traffic. The extracted features are given as input to the trained SDRK model. Algorithm 1 illustrates the data collection phase.

B. Attack Detection

The attack is detected with the trained SDRK model. Fig. 2 shows the structure of the SDRK model. Before moving to the

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:59:33 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Data Collection

input : Traffic (I_T)
output : Extracted Traffic Features (E_T)

Fog_IP ← IP of fog node; *IoT* ← IoT under Fog node
ACL ← Rules in fog node; I_T ← Incoming traffic
For Each $i \in \mathcal{I}oT$
    Gateway(i) ← Fog_IP
end
while (true) do
    Check I_T against ACL
    if (action is drop)
        Drop the packets
    else
        Mirror I_T; E_T ← Extract features from I_T
        Return E_T
end

explanation of the SDRK training, we give an overview of the algorithms, namely, DFNN and RRS-*K*-means running at the core of SDRK.

DFNN: DNN's underlying concept is a neuron that imitates the working of the human brain. As the human brain is composed of several neurons to give reasoning power, DNN strives to provide the learning capability to the machines through a Web of interconnected neurons across several hidden layers. The neurons' latent characteristic of nonlinearity enables the learner to capture the intricacies of the input–output combinations and evolve a target function to identify the attack data traffic. Of the various flavors of the DNN, we incorporate the DFNN in our model. A single layer feedforward neural network can provide any degree of accuracy, which is proved by the universal approximation theorem [23]. But this does not provide ample information on the size of the network. Through DFNN, it is possible to reduce the generalization error and have a check on the count of nodes in the hidden units.

K-Means: An attacker is not naive and has a strong domain knowledge to bring about a notable attack. As already told, the supervised form of DFNN will not suffice on a large scale. In the case of a new attack strategy, DFNN will not be able to predict it with great degrees of accuracy. Clustering has the power to classify the unknown data. Of the various types of clustering, *K*-means can converge at a fast rate to local optimum when the initial data points strongly reflect the cluster [24]. Thus, when DFNN runs on top of it, DFNN provides nearly perfect initial data points.

RRS-K-Means: The utilization of *K*-means as such will not suit the IoT scenario due to some issues. To overcome the issues, we propose a *K*-means variant named RRS-*K*-means algorithm. In this section initially, we state the reasons why *K*-means is not suitable for the IoT scenario. Then, we analyze the hardness of *K*-means for IoT and prove it to be NP-complete. Then finally, we provide the method of RRS-*K*-means along with the lower bound of the error probability of RRS-*K*-means.

The reasons why *K*-means is not suitable for IoT are as follows.

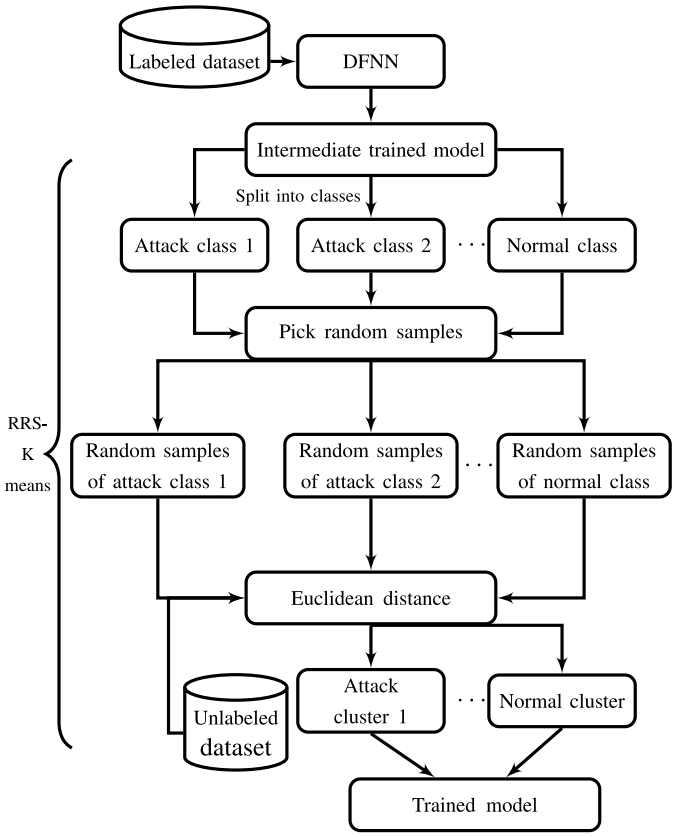


Fig. 2. SDRK training.

1) All the initial data points provided by DFNN are not similar due to diverse IoT. For instance, an object-based camera IoT in the road keeps sending a lot of images to the server during peak hours, whereas it sends few images during nonpeak hours. Similarly, each IoT is characterized by different functionality. So, a single average value of the data points as centroid is not sufficient to represent a cluster.
2) To overcome the above issue, if we resort to comparing the data instance against every data point, it leads to the wastage of computation and time cycles of the fog node. In the real world with billions of IoT, the data traffic is voluminous. Some traffic characteristics are similar. For example, a temperature sensor periodically senses the temperature and sends it to the server resulting in a constant data traffic pattern. So there are some groups of similar data points existing in the sea of data points. A single representative point of each group is enough to test the cluster membership.

On introspection of the above two points (1, 2), it is necessary to find the apt representative points of each cluster. We analyze the hardness of finding the unique representative points of the clusters and prove it to be NP-complete (Theorem 1).

Theorem 1: Finding out the unique representative points of the clusters (URP_*C*) is NP-complete.

Proof: We prove the NP-completeness of finding the representative points by reducing it to the classical dominating set problem.

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:59:33 UTC from IEEE Xplore. Restrictions apply.

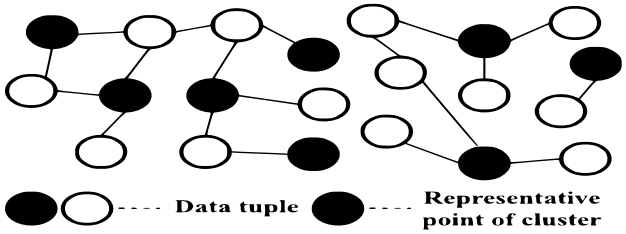


Fig. 3. Graphical view of cluster data points.

Dominating Set Problem: Given an undirected graph $G(V, E)$, find out dominating set of vertices $D \subseteq V$ such that $V - D$ is adjacent to at least one vertex of set D.

Given a set S of representative points of K clusters, a set of known test input instances T are given as input. If the instances are correctly predicted, then S is acceptable as a representative set, else S is rejected. This runs in $\mathcal{O}(|S||T|)$ time, which is polynomial. So, URP_C is in NP.

The data points of the cluster can be viewed as a graph as shown in Fig. 3. The data points act as vertices. There exists an edge between the vertices if the distance between the data points is less than a threshold (arbitrarily set).

Assume that there exists a solution to find a dominating set DS for graph $G(V, E)$. It can be seen from Fig. 3 that the corresponding solution will yield a data point from each group of nearly similar data points. Thus, if it is possible to find DS, then it is possible to find the representative points of the cluster.

Assume that there exists a solution to find the representative set of data points. Then, the solution can be applied to find a dominating set for graph G. As adding a data point to the representative set will remove the adjacent vertices which is the same as finding a dominating set.

Therefore, there is a polynomial-time reduction from dominating the set problem to URP_C. Since the dominating set is NP-complete, it follows that finding the representative points is NP-complete. ■

We propose a randomized solution named RRS-K-means to account for the NP-completeness. This involves choosing random samples (the count of random samples is an arbitrary decision made by the admin based on various factors like size of the data set, distribution of data points, and is not under the scope of this article) from the initial data points. Then, evaluate the Euclidean distance of the input data instance against the random samples. The cluster of the sample that has minimum distance to the input instance is the output. This process is repeated R times each time picking up different samples at random and the output is aggregated as a list. The final output is the cluster value which has appeared at least $\lceil (R/2) \rceil$. If there is no candidate cluster appearing at least $\lceil (R/2) \rceil$ times, then "?" is the output.

The lower bound on the error probability of the RRS-K-means with K clusters and R repeated runs is $(1 - [1/K])^{(R/2)}$ which is proved in Theorem 2.

Theorem 2: Lower bound on the error probability of RRS-K-means with K clusters and R repeated runs is $(1 - [1/K])^{(R/2)}$.

Proof: SK denotes the independent experimental run of the RRS-K-means algorithm. $\text{Prob}_l(X)$ denotes the probability that SK outputs the correct cluster for input X in exactly l out of R runs ($l < R$)

$$P(C) = \frac{1}{K} / * \text{Probability of evaluating correct cluster} * /$$

$$1 - P(C) = 1 - \left(\frac{1}{K}\right) / * \text{Probability of evaluating incorrect cluster} * /$$

$$\begin{aligned}\text{Prob}_l(X) &= \binom{R}{l} \times \left(\frac{1}{K}\right)^l \times \left(1 - \left(\frac{1}{K}\right)\right)^{R-l} \\ &= \binom{R}{l} \times \left(\frac{\frac{1}{K}}{\left(1-\left(\frac{1}{K}\right)\right)}\right)^l \times \left(1 - \left(\frac{1}{K}\right)\right)^R \\ &= \binom{R}{l} \times \left(\frac{1}{K-1}\right)^l \times \left(1 - \left(\frac{1}{K}\right)\right)^R.\end{aligned}$$

Let Y be the correct result, when X is the input. RRS-K-means outputs the result Y, if and only if Y is computed by SK at least $\lceil (R/2) \rceil$ times. The upper bound of success probability ($P(SK(X) = Y)$) of RRS-K-means running R times is as follows:

$$\begin{aligned}P(\text{SK}(X) = Y) &= 1 - \sum_{l=0}^{l=\lfloor \frac{R}{2} \rfloor} \text{Prob}_l(X) \\ &= \binom{1 - \sum_{l=0}^{l=\lfloor \frac{R}{2} \rfloor} R}{l \cdot \left(\frac{1}{K-1}\right)^l \cdot \left(1 - \left(\frac{1}{K}\right)\right)^R} \\ &= \binom{1 - \left(1 - \left(\frac{1}{K}\right)\right)^R \cdot \sum_{l=0}^{l=\lfloor \frac{R}{2} \rfloor} R}{l \cdot \left(\frac{1}{K-1}\right)^l} \\ &< \binom{1 - \left(1 - \left(\frac{1}{K}\right)\right)^R \cdot \sum_{l=0}^{l=\lfloor \frac{R}{2} \rfloor} \lfloor \frac{R}{2} \rfloor}{l \cdot \left(\frac{1}{K-1}\right)^l} \\ &< 1 - \left(1 - \left(\frac{1}{K}\right)\right)^R \cdot \left(1 + \frac{1}{K-1}\right)^{\frac{R}{2}} \\ &< 1 - \left(\frac{K-1}{K}\right)^R \cdot \left(\frac{K}{K-1}\right)^{\frac{R}{2}} \\ &< 1 - \left(\frac{K-1}{K}\right)^{\frac{R}{2}} \\ &< 1 - \left(1 - \frac{1}{K}\right)^{\frac{R}{2}}.\end{aligned}$$

Thus, the lower bound on the error probability of the RRS-K-means algorithm is observed to be

$$\left(1 - \frac{1}{K}\right)^{\frac{R}{2}}.$$

Hence proved Theorem 2. ■

If ζ is the desired error probability and K is the number of clusters, then the number of independent experimental runs R of the RRS-K-means algorithm is $([2 \log_b \zeta]/[\log_b(1 - [1/K])])$ which is proved by Theorem 3.

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:59:33 UTC from IEEE Xplore. Restrictions apply.

Theorem 3: If ζ is the desired error probability and K is the number of clusters then the number of independent experimental runs R of the RRS-K-means algorithm is $([2\log_b \zeta]/[\log_b(1-[1/K])])$.

Proof: Success probability $[P(SK(X) = Y)]$ of RRS-K-means running R times is as follows:

$$P(\mathrm{SK}(X) = Y) < 1 - \zeta$$

$$P(\mathrm{SK}(X) = Y) < 1 - \left(1 - \frac{1}{K}\right)^{\frac{R}{2}} / * \text{Refer theorem 2} * /$$

$$\implies \zeta = \left(1 - \frac{1}{K}\right)^{\frac{R}{2}}.$$

Taking $\log_b$ on both sides,

$$\log_b \zeta = \log_b\left(1 - \frac{1}{K}\right)^{\frac{R}{2}} = \frac{R}{2} \cdot \log_b\left(1 - \frac{1}{K}\right)$$

$$2 \cdot \log_b \zeta = R \cdot \log_b\left(1 - \frac{1}{K}\right)$$

$$R = \frac{2 \cdot \log_b \zeta}{\log_b\left(1 - \frac{1}{K}\right)}.$$

Hence proved Theorem 3. ■

For instance, consider the following examples that evaluate the number of runs R required when error probability ζ and the number of clusters K are given:

Example 1: $\zeta = 0.01$, $K = 10$

$$R = \frac{2 \cdot \log_{10} 0.01}{\log_{10}\left(1 - \frac{1}{10}\right)} = 87.417 \approx 87.$$

Example 2: $\zeta = 0.01$, $K = 1000$

$$R = \frac{2 \cdot \log_{10} 0.01}{\log_{10}\left(1 - \frac{1}{1000}\right)} = 9205.734 \approx 9206.$$

From Theorem 3 and the above examples, it can be seen that the number of runs R is a constant, when its input values ζ and K are constants. If D is any deterministic algorithm that puts input X in the appropriate cluster, then the RRS-K-means algorithm taking same input X, with error bounds $(1-[1/K])^{[R/2]}$ (Theorem 2) is as efficient as D. Hence

$$\mathrm{Time}(\mathrm{RRS} - \mathrm{K} - \mathrm{means}(X)) \in \mathcal{O}(\mathrm{Time}(D(X)).$$

Now, we explain the steps involved in the training of the SDRK model.

SDRK Training: Algorithms 2 and 3 give detailed steps for SDRK training. Three sets of data are provided as input for training. They are labeled as training data set TD_L, test data set TD, and unlabeled training data set $\mathrm{TD}_{\mathrm{UL}}$. A gist of the overall flow of training the SDRK model is as follows.

Step 1: The parameters—the number of layers, count of neurons in each layer, activation function across each layer, count of the epoch, learning rate, and weight matrix are initialized. TD_L is given as input to the DFNN.

Step 2: Check if error at the final epoch is at an acceptable level. If yes, return intermediate trained model TM_I and proceed to step 3. Else, reinitialize the parameters, and start from step 1.

Algorithm 2: Training of SDRK

input : N labeled training instances TD_L
output : *Intermediate_Trained_Model* (TM_I)

Initialize layers (l), count of neurons (H), bias, weight (W), learning rate (η), epoch (E); *accuracy* $\leftarrow \varnothing$
observed $\leftarrow$ *Input* TD_L *to DFNN*
accuracy $\leftarrow$ *(#(actual==observed)* $\div$ *N)* $\times$ *100*
if (accuracy is not satisfactory)
Change parameters - E, η, H, W; Repeat step 3
else
return TM_I

Algorithm 3: Training of SDRK (Contd.)

input : TM_I; D test instances TD(x_{tt},y_{tt}), where x_{tt}=[x_{tt1},...,x_{ttA}] & y_{tt} - class; M unlabeled training instances $TD_U(x_{tu})$, where x_{tu}=[x_{tu1},...,x_{tuA}] /* A= #(features) */
output : Trained_Model

$R \leftarrow$ RRS runs; *Thres* $\leftarrow$ distance threshold; *Euclid* $\leftarrow \varnothing$; /* Euclid is the set of distance */
pred $\leftarrow TM_I(x_{tt})$ /* Input test dataset */
for i = 1 to D:
if ($pred_i$ == $TD(y)_i$) $TD_L \leftarrow TD_L \cup TD(x)_i$
else $TD_U \leftarrow TD_U \cup TD(x)_i$
end for
K $\leftarrow$ #(classes of TD_L); Create K clusters
Iterate over the TD_L and put the datapoints in clusters
for i = 1 to #(TD_U):
clus_output $\leftarrow \varnothing$ /* RRS-KMeans output */
$R > \frac{2\cdot\log_b \zeta}{\log_b(1-\frac{1}{K})}$ /* See theorem 3 */
for r = 1 to R:
S $\leftarrow$ initial seed for generating random numbers
RN $\leftarrow$ set of generated random numbers with S
S $\leftarrow$ samples from TD_L at the indexes of set RN
for j = 1 to #(RN):
Dist $\leftarrow$

$$\sqrt{(S_j^{[1]} - TD_{Ui}^{[1]})^2 + \ldots + (S_j^{[A]} - TD_{Ui}^{[A]})^2}$$

Euclid $\leftarrow$ Euclid $\cup$ Dist
end for
sort Euclid /* Sort the distance */
euclid1 $\leftarrow$ top(sorted Euclid)
if (euclid1 < Thres)
cluster $\leftarrow$ class(euclid1)
else
cluster $\leftarrow$?
clus_output $\leftarrow$ clus_output $\cup$ cluster
end for
final_cluster $\leftarrow$ class occurring at least $\lceil \frac{R}{2} \rceil$ times
if (final_cluster == $\varnothing$) then final_cluster_output $\leftarrow$?
if (final_cluster == ?) then Create alert
if (final_cluster != ?) then Put TD_{Ui} to final_cluster
end for
return Trained_Model

Step 3: Input TD to TM_I. Pick correctly labeled tuples of TD (T). $\mathrm{TD}_L \leftarrow \mathrm{TD}_L \cup T$. $\mathrm{TD}_{\mathrm{UL}} \leftarrow \mathrm{TD}_{\mathrm{UL}} \cup (\mathrm{TD} - T)$.

Step 4: Count the number of predictor classes of TD_L (K). Create K clusters and label them with the predictor class names. Group TD_L based on their class names and put them

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:59:33 UTC from IEEE Xplore. Restrictions apply.

Algorithm 4: Attack Detection

input : Extracted Traffic Features (E_T) from algo 1
output : Attack status (A_S)

$A_S \leftarrow$ *Trained_Model*(E_T)
if (A_S == **normal) then** Continue with normal operation
if (A_S == **attack_class) then** Return A_S
if (A_S == **?) then** Create alert. Return A_S

Algorithm 5: Attack Mitigation

input : Attack status(A_S)
output : ACL Rule

if (A_S == **attack_class)**
Set drop rules to the MAC-IP that sent packets
Set random timer(RT)
if (RT expires) then Revoke the drop rule
if (A_S == **?)**
Set drop rules to the MAC-IP that sent packets
Set small random timer(RT_S)
if (RT_S **expires) then** Revoke the drop rule

in respective clusters. These points act as initial data points for the clusters.

Step 5: Compute the number of experimental runs: $R \leftarrow ([2 \cdot \log_b \zeta]/[\log_b(1-[1/K])])$ (Theorem 3). Pick RN_i random samples from clusters for each run $i = 1, 2, \ldots, R$.

Step 6: Compute the nearest cluster for every $\mathrm{TD_{UL}}$ instance, with RN_i random samples taken as reference data points during each run $i = 1, 2, \ldots, R$. Add the unlabeled instance to the respective cluster if at least half of the runs output that cluster and the Euclidean distance does not cross a threshold. Else output "?."

Step 7: If the output is "?," then generate alert and send a detailed report of that instance to the admin.

Step 8: The admin decides the nature of the instance and decides whether to add it to a new or an existing cluster.

Step 9: Return *Trained_Model*.

The collected data are normal, then given as input to the *Trained_Model*. Algorithm 4 illustrates the attack detection phase. If the model classifies the traffic to be normal, then the normal operation continues. If the model classifies the traffic as an attack, then the mitigation module is triggered. Else if the model outputs "?," then an alert is generated, the mirrored traffic is sent to the admin and the mitigation module is triggered.

C. Attack Mitigation

Algorithm 5 illustrates the attack mitigation module. When the mitigation module is triggered, if the class is one among the attacks, the drop rule is set on the fog node to the MAC-IP address pair of the IoT that has sent the packets during the captured time. Also, a random timer is set. After the timer expires, the drop rule is revoked from the ACL. The timer is used to avoid the legitimate IoT from getting denied the service, because at the capture time instance, legitimate traffic may also be passing through the fog node. The timer should be random so that the attacker cannot deduce the mitigation time strategy. The captured traffic is sent to the admin, who probes into it and updates the model if necessary. If the mitigation module receives "?" as input, the drop rule is set for a relatively small random time because the model is unsure of the class of traffic. The range of the timer is predetermined by the admin. Admin examines the unsure traffic. If it is attack traffic, the drop rule is set. Else, no rule is set. The admin updates the model with the new observed traffic instance if necessary.

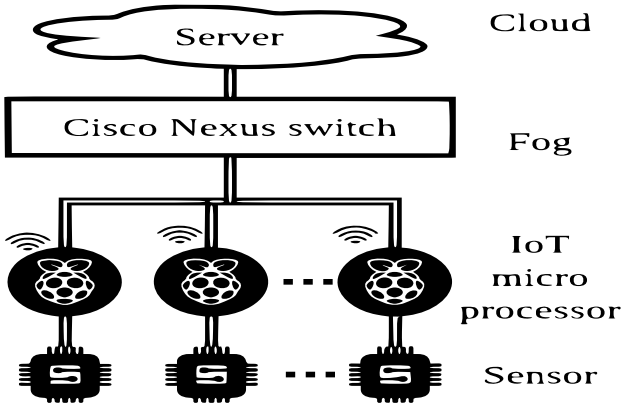


Fig. 4. Testbed.

V. Evaluation

A. Testbed Setup

Fig. 4 depicts the high-level connectivity of our IoT–fog–cloud testbed [25]. Cisco Nexus switch 5672UP is the fog node. Its features are—Cisco NX-OS operating system; programmable; 16 fixed SFP+ UP, 32 fixed 1/10GE SFP+, and 6 fixed 40GE QSFP+ ports; 1.44 Tb/s and 1071 Mpps for 64-B packets; Cisco switched port analyzer (SPAN); 47–63-Hz operating frequency, etc. Our cloud comprises of—Quad-core Intel Xeon CPU E5620@2.40 GHz; Xen PVM hypervisor; 36 GB RAM; 3*600 GB HDD; and Debian 7.0 OS. The access point used is Edimax BR-6428NS V4 N300. We use the Raspberry Pi Model 3 B with—1.2-GHz 64-B quad-core ARMv8 processor; 1-GB RAM size; Raspbian Stretch OS; and 64-GB memory card as the microprocessor for IoT. Totally 30 IoT is set with Raspberry Pi connected to various sensors, such as the pi matrix, camera built-in pi matrix, DHT11 temperature and humidity sensor, TCRT 5000 obstacle avoidance infrared tracking sensor, PIR motion detector, etc. The traffic generated from the IoT is mirrored using the SPAN feature of the Cisco Nexus switch. The traffic is processed and given as input to the SDRK model. The programmable feature of our fog node is used to install ACL rules for mitigating the attack.

B. Results and Discussion

For testing our model, we ran two experimental drives.

1) We generate the DD attack in the testbed and elicit a data set to train the SDRK model.
2) We test the SDRK model with the benchmark NSL-KDD data set. We run this experiment for three reasons: a) to ascertain the effectiveness of the SDRK model on the

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:59:33 UTC from IEEE Xplore. Restrictions apply.

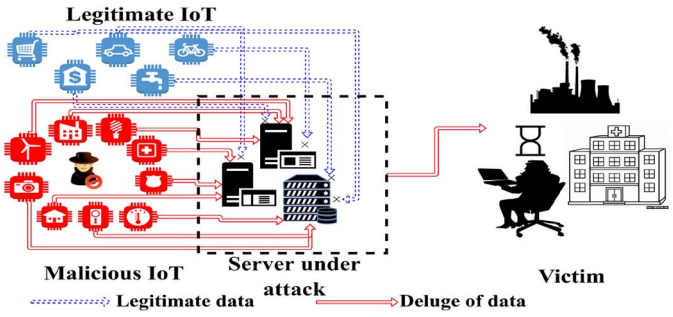


Fig. 5. DD attack model.

benchmark data set; b) NSL-KDD has diverse attack vectors, hence it will be a good data set for evaluating SDRK; and c) state-of-the-art solutions [3], [6]–[9], [21] utilize NSL-KDD data set to evaluate their models, so to have a fair comparison with them, we test with the NSL-KDD data set.

1) DD Attack (Attack Model): Fig. 5 illustrates the DD attack model. The DD attack is an ilk of the DDoS attack. The malicious attackers generate a flood of sensed data to bring down the whole network by consuming the resources. IoT has two prime data generation models: 1) the IoT is set up to sense the environment once in a time cycle and send the data to a server and 2) the IoT stays idle sensing the environment and is triggered to send data only during an occurrence of an event. (For instance, fire alarm smoke sensors create an alert and sends data when it senses smoke.) The DD attack can be generated by attackers using both time-based and event-based data generation models.

In the time-based attack model, the attacker has to bring the IoT under his control. The attacker can do it through botnets or any other means of the remote control. (We are not going into further details of botnets and rather focus on the attack propagation.) The attacker modifies the period of sensing in IoT to very low values. For instance, when the sensing period is modified to 0.0001 s, then one IoT itself generates 600 000 packets/s. When several IoT in a network field are hacked, a deluge of packets is generated. This will consume the constrained resources of the IoT network like power, bandwidth, etc., and lead to downtime. For instance, in a hospital, IoT is placed on the human body for monitoring. When a DD attack is launched it blocks the service to the critical patients creating a danger to life.

In an event-based attack, the attacker triggers the event that will generate a deluge of data. For instance, in an infrared motion sensor, it sends data when an object falls in its sensing range. The attacker can place an object that will intercept the infrared rays, which will eventually generate data unnecessarily. This way is not as effective as a time-based attack model in bringing down a whole network, but it can surely create some havoc like consumption of power, false alarm, filling server with spam data, etc.

Data Set: To gather the data set, we generate a DD attack on the testbed. Malicious IoT is selected randomly to get a variety of attack traffic and not monotonous attack traffic. When an IoT is chosen to be malicious, the DD attack script is triggered. During the rest of the times, the IoT runs the normal data generation script. The traffic is mirrored on the SPAN ports of the fog switch. The data processing module captures the traffic once every time window (0.5 s chosen arbitrarily) and extracts necessary statistical feature data as given in Table II.

TABLE III
TRAINING PARAMETERS

Parameter	Value
Count of hidden layers	7
Epoch	800
Count of neurons across hidden layers	100,150,200,150,150,100,50
Activation function across layers	sigmoid
Learning rate	0.06

PC characterizes the arrival rate of the incoming traffic. LB denotes the bandwidth consumed by the traffic. We have already said that the IoT network normally generates a high amount of data traffic due to its size and hence it is difficult to distinguish if it is a normal or DD attack. To distinguish it, the statistical features, such as A, WA, SD, M, and R aids in indicating the average size of the traffic generated in the current time window and variation of the incoming traffic across the previous time windows. (On trial-and-error basis, we find that WA calculated across 14 previous time frames gives better accuracy. Therefore, $t = 14$ is set for our experiment.) CV, CR, CV_1, and CR_1 capture the traffic conditions in cases of traffic deluge generated by spoofed IP [26]. E and ID_j are known indicators of the DDoS attack [27]. On trial and error basis, we find that information distance measure between the current time window and the previous ten time windows gives better accuracy. Thus, each data set tuple had 22 attributes.

SDRK Training: A total of 115 200 data set tuples are generated. Of them, 11 520 tuples are labeled and given as initial input. The remaining data tuples and the incorrectly classified tuples are given as input to the intermediate trained model.

To evaluate the optimal number of hidden layers, learning rate, neurons in each hidden layer, activation function used in each layer, and epochs, various combinations of trial and error runs are executed. During each run, the root-mean-square error (RMSE) is evaluated. The parameters which give minimal RMSE are fixed as the optimal value and used for training. Table III shows the optimal parameter values.

Performance Evaluation:

Degree of Correctness: We inject the DD attack at various time instances during experimentation and vary the count of malicious IoT. The degree of correctness is evaluated and the results are compared with other ML models.

The metrics are as follows (aa'—true postive; $n'a$—false postive; $a'n$—false negative; and $n'n$—true negative; where, $a \leftarrow$ predicted as attack; $n \leftarrow$ predicted as normal; $a' \leftarrow$ actually an attack; and $n' \leftarrow$ actually normal).

1) *Accuracy (ACC):* This indicates the percentage of correctly detected network traffic instances i.e., if the network is subject to attack or not

$$\text{ACC} = \left(\frac{aa' + n'n}{aa' + n'n + a'n + n'a}\right) * 100. \quad (1)$$

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:59:33 UTC from IEEE Xplore. Restrictions apply.

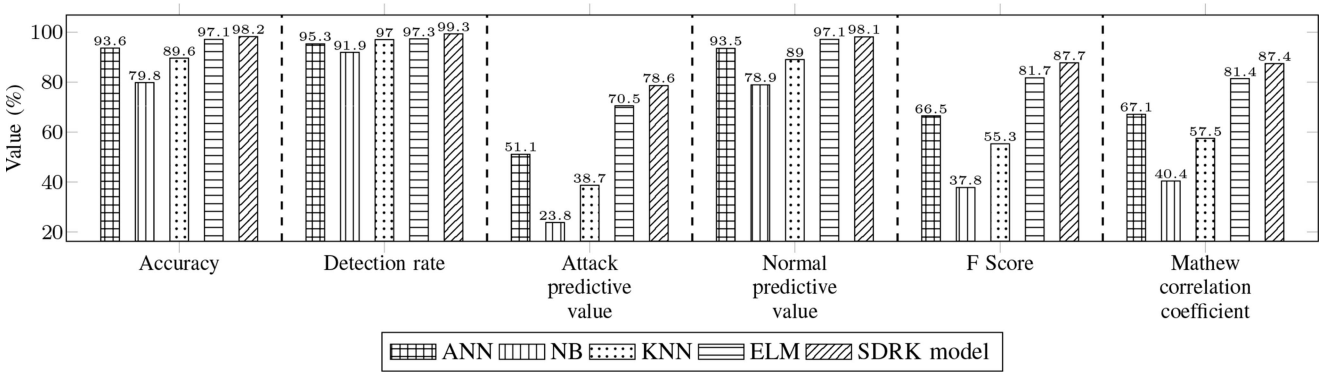


Fig. 6. Degree of correctness.

2) *Detection Rate (DR):* It denotes the percentage of correctly detected attack traffic instances among the total attack traffic instances

$$\mathrm{DR} = \left(\frac{aa'}{aa' + a'n}\right) * 100. \tag{2}$$

3) *Attack Predictive Value (APV):* It denotes the percentage of the correctly detected attack traffic instances among the instances predicted as attack

$$\mathrm{APV} = \left(\frac{aa'}{aa' + n'a}\right) * 100. \tag{3}$$

4) *Normal Predictive Value (NPV):* It denotes the percentage of the correctly detected normal traffic instances among the actual normal traffic instances

$$\mathrm{NPV} = \left(\frac{nn'}{n'a + n'n}\right) * 100. \tag{4}$$

5) *F Score (F):* This metric is a combined form of DR and APV. This will denote the overall percentage of the correct detection of attack traffic instances

$$F = \left(\frac{2 * aa'}{(2 * aa') + n'a + a'n}\right) * 100. \tag{5}$$

6) *Mathew Correlation Coefficient (M):* This denotes the overall correlation coefficient between actual traffic status and predicted traffic status. MCC has a value between -100% and 100%. The model is better, as MCC tends to 100

$$M = \left(\frac{(aa' * n'n) - (n'a * a'n)}{\sqrt{(aa'+a'n)(aa'+n'a)(n'a+n'n)(n'n+a'n)}}\right) * 100. \tag{6}$$

Fig. 6 illustrates the degree of correctness of the SDRK model and other ML algorithms, such as artificial neural network (ANN), NB, K-nearest neighbor (KNN), and ELM. It can be seen from Fig. 6 that the SDRK model is better in detecting the DD attack.

Response Time: Our model is implemented in the cloud to evaluate if the model installed in the fog device is efficient than its deployment in the cloud. Fig. 7 shows the time taken to respond to the DD attack in the fog device and in the cloud (i.e., time taken to deduce the malicious IoT and set ACL rules in the fog device, after mirroring DD traffic). We achieve

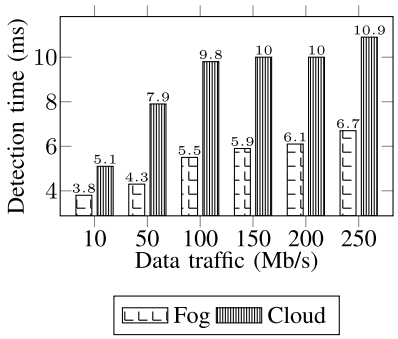


Fig. 7. Detection time—fog and cloud.

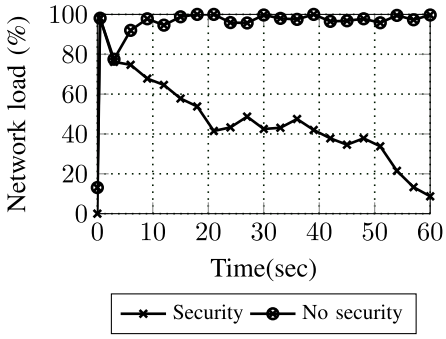


Fig. 8. Network load.

a dip in response time by at least 34%. The reason for the reduced response time when deployed at the fog node is due to both edge detection and the increased processing power of our fog node. Therefore, based on the device you choose as the fog node, the percent of the dip will vary. But there will be a notable dip in response time and faster attack mitigation when compared to the deployment of the security module in the cloud. The response time when deployed at cloud increases because the cloud processes all the traffic from fog nodes and, the cloud sends the ACL rules for mitigation to the appropriate fog nodes.

Network Load: Fig. 8 illustrates the network load on the network link during the attack when the model is deployed and when not deployed on the fog node. The multirouter traffic grapher (MRTG) tool is used to evaluate the network load. It can be seen from Fig. 8 that even when the DD attack is

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:59:33 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
DEGREE OF CORRECTNESS—NSL-KDD DATA SET

Work	Detection strategy	Accuracy (%)	Detection rate (%)	Attack predictive value (%)	Normal predictive value (%)	F score (%)	Mathew correlation coefficient (%)	Training time (s)	Testing time (s)	Total time (s)
[18]	ELM	94.45	95.73	95.47	88.66	96.11	84.39	602.59	19.89	622.48
[3]	Online Sequential ELM	97.36	96.02	96.95	97.69	96.93	94.62	678.44	21.87	700.31
[9]	ELM Fuzzy C Means	94.61	94.52	93.06	94.66	93.78	89.03	793.05	32.56	825.61
[6]	Deep learning	98.28	96.75	98.46	97.46	98.39	97.01	1915.19	18.42	1933.61
[7]	LDA, Naive Bayes, K-NN	92.23	86.89	89.23	86.96	87.93	80.45	516.23	30.69	546.92
[8]	Genetic algorithm & DBN	99.47	99.72	99.29	99.68	99.61	99.24	2011.23	18.02	2029.25
[10]	SVM	95.18	96.08	96.79	93.59	95.53	88.26	603.91	40.81	644.72
[21]	Bat algorithm & Random Forest	99.42	99.19	99.51	95.17	97.29	98.13	3128.55	23.62	3152.17
SDRK	SDRK	99.78	99.83	99.62	99.79	99.72	99.44	1986.32	6.59	1992.91

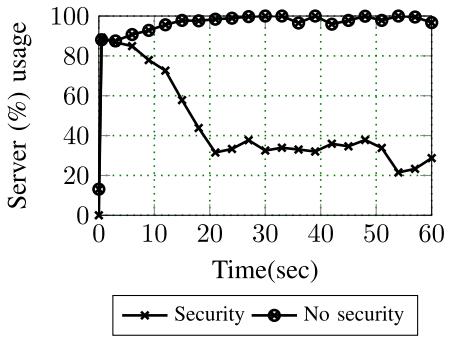


Fig. 9. Server processor usage.

generated, the security module mitigates the attack by setting drop rules.

Server Processor Usage: In the testbed, the data generated from the IoT sensors reach the server in the cloud where data processing modules are running. We code scripts to monitor the server processing usage once every 0.5 s. Fig. 9 shows the usage accounted when the DD attack is launched in two scenarios: 1) the security model is in the running state in fog device and 2) when no security model is running. As seen from Fig. 9, the security model running in the fog drops the malicious traffic and prevents the consumption of processing cycles in the server.

2) NSL-KDD Data Set: To ascertain the efficiency of the SDRK model, we further test the model with a benchmark NSL-KDD network intrusion detection data set. We compare the results with state-of-the-art solutions in [3], [6]–[10], [18], and [21]. The NSL-KDD data set comprises of 41 attributes, such as duration, urgent, src_bytes, dst_bytes, count, etc., and one traffic class label [9]. The data set has normal traffic instances and various attack traffic class instances, such as teardrop, land, rootkit, port sweep, SPY, IMAP, etc., which falls under any one of the four attack categories, namely, DDoS, probing, user-to-root, and remote-to-local. The attributes are either symbolic or numeric. To process the symbolic attributes, the indicator/dummy variable approach followed by clustering as in [9] is used. The training data set KDDTrain is split randomly into an unlabeled and labeled data set. 90% of the training data set are put in the unlabeled category with the class label field removed and the rest of the data set is put in the labeled data set category [9].

Performance Evaluation: We compare using the degree of correctness metrics (accuracy, detection rate, attack predictive value, normal predictive value, F Score, and Mathew correlation coefficient) given in Section V-B1 using (1)–(6).

Table IV gives the degree of correctness metric values evaluated for the NSL-KDD test data set. It can be seen from Table IV that the SDRK model is better than other models. The training and testing time are also given in Table IV. The shallow ML and ELM-based models take less training time than deep ML models. But when looking at the accuracy, deep ML models are better than the rest of the models. The testing time of SDRK is less than other models because SDRK compares with random samples from the cluster over a finite number of runs. This shows that SDRK is a better model for latency-critical IoT use cases. Though the accuracy of SDRK shows less improvement than [8] and [21], SDRK has better testing time than [8] and [21].

VI. CONCLUSION

In this article, we proposed a methodology for detecting and mitigating intrusion in the IoT network. The methodology is based on a novel semisupervised learning model named SDRK. We test the methodology in IoT–fog testbed. The SDRK model placed in the fog node can detect the attack faster than when placed in the cloud, which is very much necessary in cases of latency-critical IoT services. As the SDRK model is semisupervised, it also overcomes the issues in labeling a huge data set. When compared to state-of-the-art solutions, our model shows better accuracy in detecting the security breaches.

Our planned works in line are to probe into other semisupervised models to bring about improvement in the accuracy of detecting attacks. Though the attack detection time of the trained model is less than state-of-the-art solutions, the training time is higher than the ELM-based models. Hence, when the model has to be retrained in an online fashion in the real-time network, it may take time. Our future work will be to find ways

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:59:33 UTC from IEEE Xplore. Restrictions apply.

to optimize the retraining time to fine-tune the performance of the model for the real-time network. In this article, we assume that the fog nodes are not subject to attack, but there are possibilities that the attacker can hack in and control the fog nodes. Our next step will be to verify if the SDRK model can analyze that the fog node is the target of the attackers through the incoming traffic. We also plan to generate a comprehensive set of IoT network attacks in the testbed and evaluate the accuracy of attack detection.

REFERENCES

[1] N. Neshenko, E. Bou-Harb, J. Crichigno, G. Kaddoum, and N. Ghani, "Demystifying IoT security: An exhaustive survey on IoT vulnerabilities and a first empirical look on Internet-scale IoT exploitations," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2702–2733, 3rd Quart., 2019.

[2] D. Abbasinezhad-Mood, A. Ostad-Sharif, and M. Nikooghadam, "Novel anonymous key establishment protocol for isolated smart meters," *IEEE Trans. Ind. Electron.*, vol. 67, no. 4, pp. 2844–2851, Apr. 2020.

[3] S. Prabavathy, K. Sundarakantham, and S. M. Shalinie, "Design of cognitive fog computing for intrusion detection in Internet of Things," *J. Commun. Netw.*, vol. 20, no. 3, pp. 291–298, Jun. 2018.

[4] M. Haus, M. Waqas, A. Y. Ding, Y. Li, S. Tarkoma, and J. Ott, "Security and privacy in device-to-device (D2D) communication: A review," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 2, pp. 1054–1079, 2nd Quart., 2017.

[5] A. Abeshu and N. Chilamkurti, "Deep learning: The frontier for distributed attack detection in fog-to-things computing," *IEEE Commun. Mag.*, vol. 56, no. 2, pp. 169–175, Feb. 2018.

[6] A. A. Diro and N. Chilamkurti, "Distributed attack detection scheme using deep learning approach for Internet of Things," *Future Gener. Comput. Syst.*, vol. 82, pp. 761–768, May 2018.

[7] H. H. Pajouh, R. Javidan, R. Khayami, A. Dehghantanha, and K.-K. R. Choo, "A two-layer dimension reduction and two-tier classification model for anomaly-based intrusion detection in IoT backbone networks," *IEEE Trans. Emerg. Topics Comput.*, vol. 7, no. 2, pp. 314–323, Apr.–Jun. 2019.

[8] Y. Zhang, P. Li, and X. Wang, "Intrusion detection for IoT based on improved genetic algorithm and deep belief network," *IEEE Access*, vol. 7, pp. 31711–31722, 2019.

[9] S. Rathore and J. H. Park, "Semi-supervised learning based distributed attack detection framework for IoT," *Appl. Soft Comput.*, vol. 72, pp. 79–89, Nov. 2018.

[10] S. U. Jan, S. Ahmed, V. Shakhov, and I. Koo, "Toward a lightweight intrusion detection system for the Internet of Things," *IEEE Access*, vol. 7, pp. 42450–42471, 2019.

[11] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: MIRAI and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017.

[12] O. Salman, I. Elhajj, A. Chehab, and A. Kayssi, "IoT survey: An SDN and fog computing perspective," *Comput. Netw.*, vol. 143, pp. 221–246, Oct. 2018.

[13] I. U. Din, H. Asmat, and M. Guizani, "A review of information centric network-based Internet of Things: Communication architectures, design issues, and research opportunities," *Multimedia Tools Appl.*, vol. 78, no. 21, pp. 30241–30256, 2019.

[14] J. Lin, W. Yu, N. Zhang, X. Yang, H. Zhang, and W. Zhao, "A survey on Internet of Things: Architecture, enabling technologies, security and privacy, and applications," *IEEE Internet Things J.*, vol. 4, no. 5, pp. 1125–1142, Oct. 2017.

[15] K. Kalkan and S. Zeadally, "Securing Internet of Things with software defined networking," *IEEE Commun. Mag.*, vol. 56, no. 9, pp. 186–192, Sep. 2018.

[16] J. Carlquist, "Evaluating the use of ICN for Internet of Things," M.S. thesis, Dept. Inf. Technol., Uppsala Universitet, Uppsala, Sweden, 2018.

[17] R. K. Naha *et al.*, "Fog computing: Survey of trends, architectures, requirements, and research directions," *IEEE Access*, vol. 6, pp. 47980–48009, 2018.

[18] R. Kozik, M. Choraś, M. Ficco, and F. Palmieri, "A scalable distributed machine learning approach for attack detection in edge computing environments," *J. Parallel Distrib. Comput.*, vol. 119, pp. 18–26, Sep. 2018.

[19] R. ManjuParkavi, M. Shanthi, and M. C. Bhuvaneshwari, "Extreme learning machine algorithm and its application," in *Proc. 2nd Int. Conf. Latest Trends Eng. Sci. Humanities Manag.*, 2017, pp. 1–9.

[20] R. Salakhutdinov and H. Larochelle, "Efficient learning of deep Boltzmann machines," in *Proc. 13th Int. Conf. Artif. Intell. Stat.*, 2010, pp. 693–700.

[21] J. Li, Z. Zhao, R. Li, and H. Zhang, "AI-based two-stage intrusion detection for software defined IoT networks," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2093–2102, Apr. 2019.

[22] T. G. Nguyen, T. V. Phan, B. T. Nguyen, C. So-In, Z. A. Baig, and S. Sanguanpong, "SeArch: A collaborative and intelligent NIDS architecture for SDN-based cloud IoT networks," *IEEE Access*, vol. 7, pp. 107678–107694, 2019.

[23] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359–366, 1989.

[24] C. Chang, W. Liao, Y. Chen, and L. Liou, "A mathematical theory for clustering in metric spaces," *IEEE Trans. Netw. Sci. Eng.*, vol. 3, no. 1, pp. 2–16, Jan.–Mar. 2016.

[25] N. Ravi and S. M. Shalinie, "TeFENS: Testbed for experimenting next-generation-network security," in *Proc. IEEE 5G World Forum (5GWF)*, Silicon Valley, CA, USA, 2018, pp. 204–209.

[26] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "KITSUNE: An ensemble of autoencoders for online network intrusion detection," 2018. [Online]. Available: arXiv:1802.09089.

[27] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "An empirical evaluation of information metrics for low-rate and high-rate DDoS attack detection," *Pattern Recognit. Lett.*, vol. 51, pp. 1–7, Jan. 2015.



Nagarathna Ravi (Student Member, IEEE) received the B.Tech. degree in information technology from Madras Institute of Technology, Anna University, Chennai, India, and the M.E. degree in computer science and engineering from the Thiagarajar College of Engineering, Madurai, India, where she is currently pursuing the Ph.D. degree.

She has published around ten papers. Her research interests are SDN and IoT security.



S. Mercy Shalinie (Senior Member, IEEE) received the B.E. degree from Annamalai University, Chidambaram, India, in 1989, the M.E. degree from Coimbatore Institute of Technology, Coimbatore, India, in 1991, and the Ph.D. degree from Madurai Kamaraj University, Madurai, India, in 1999.

She is a Professor with the CSE Department, Thiagarajar College of Engineering, Madurai. She was a Postdoctoral Fellow with the University of California at Irvine, Irvine, CA, USA, in 2008. She has published over 200 research papers. Her areas of research interests include machine learning and security systems.

Authorized licensed use limited to: b-on: ISCTE. Downloaded on October 10,2026 at 10:59:33 UTC from IEEE Xplore. Restrictions apply.